

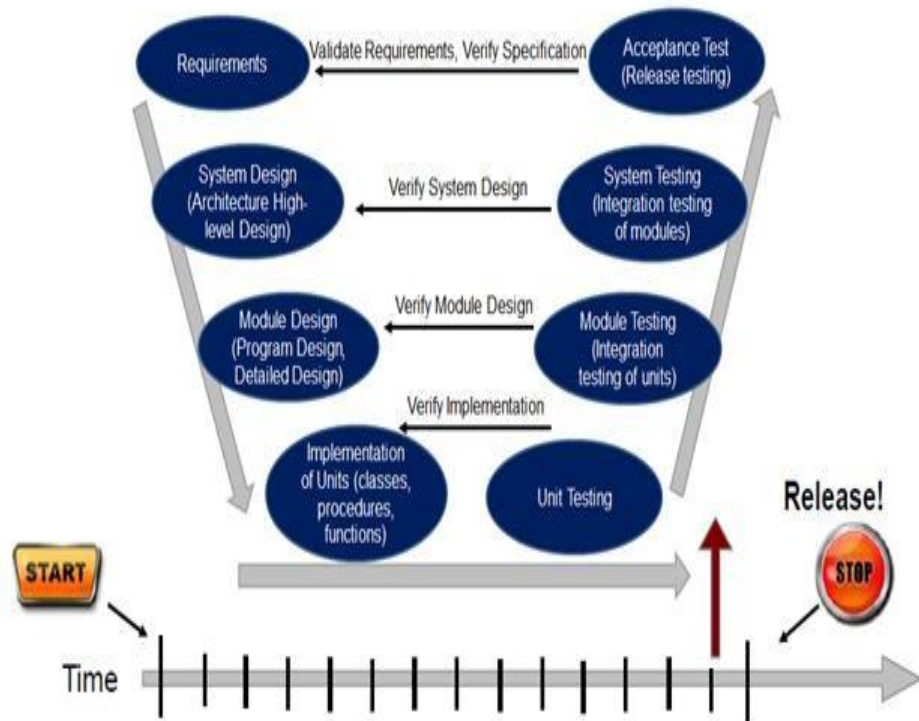


Software Engineering

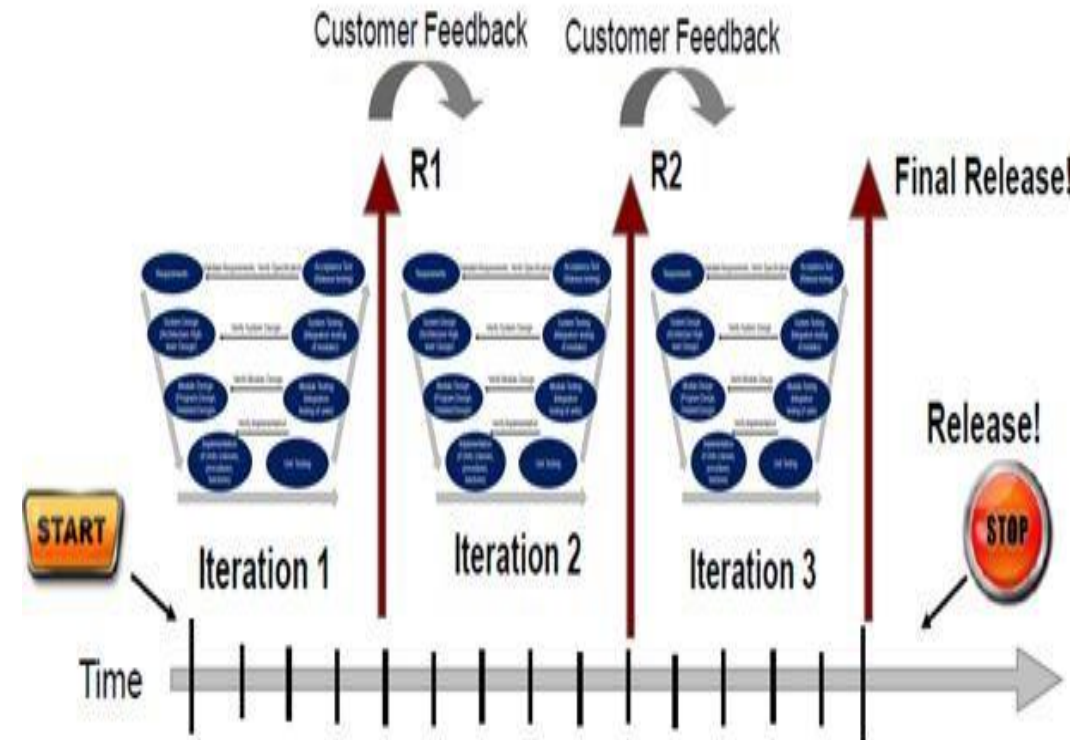
Tahir Farooq/Tehreem Aslam
FAST-NU



Incremental Development



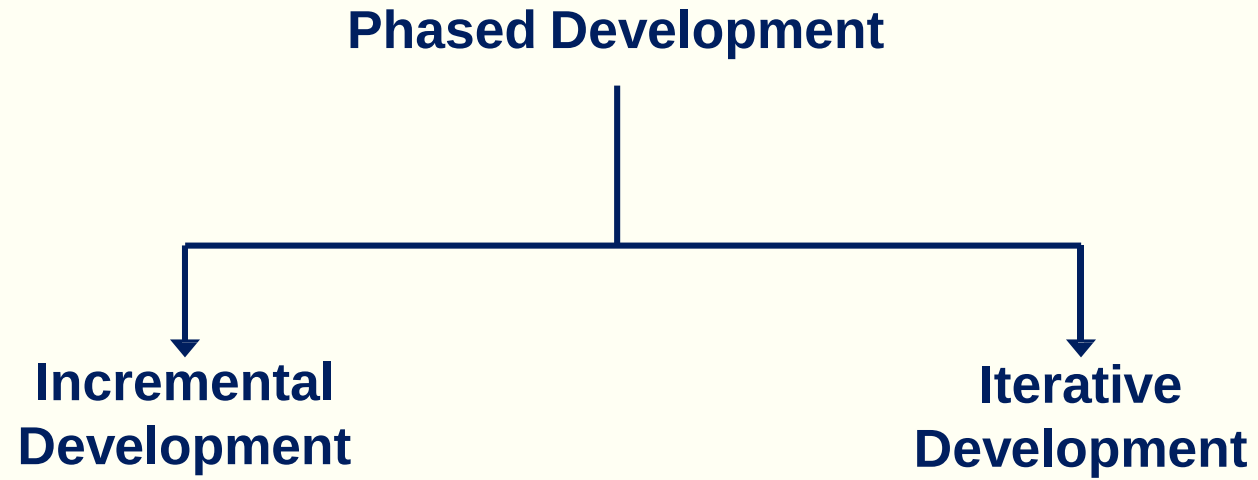
Iterative Development



Phased Development

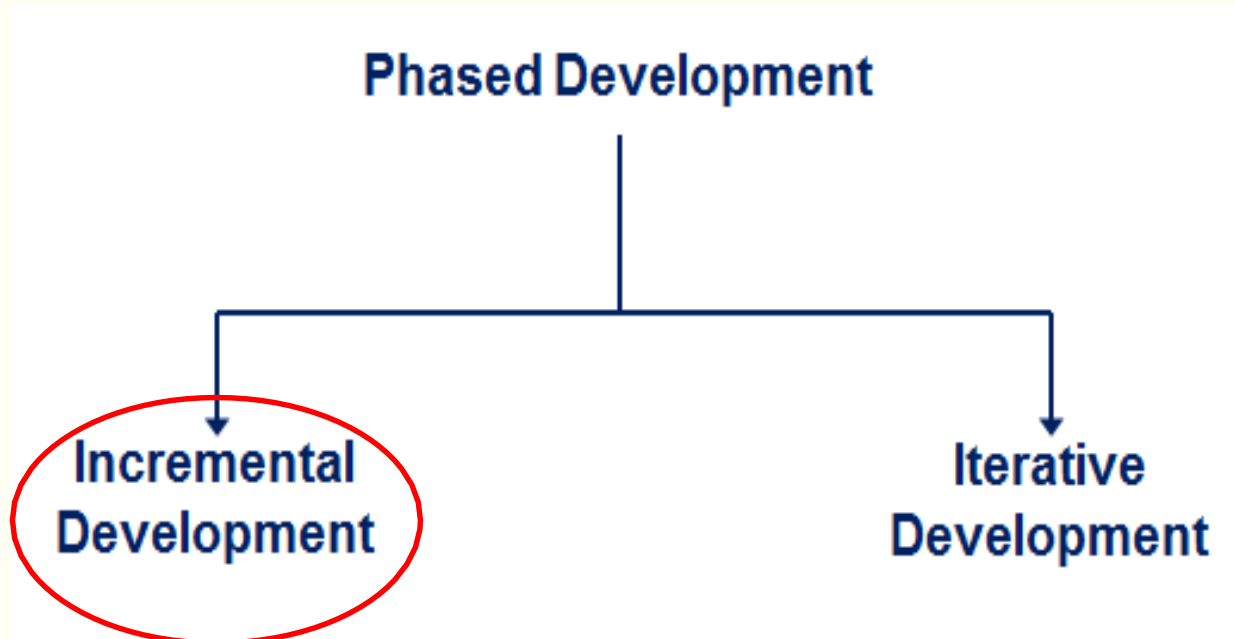
- Design a system so it can be delivered in pieces, letting users have some functionality while the rest is under development
- There are usually two or more systems in parallel:
 - a) The **operational** or **production** system in use by customers
 - b) The **development system** which will replace the current release

Phased Development

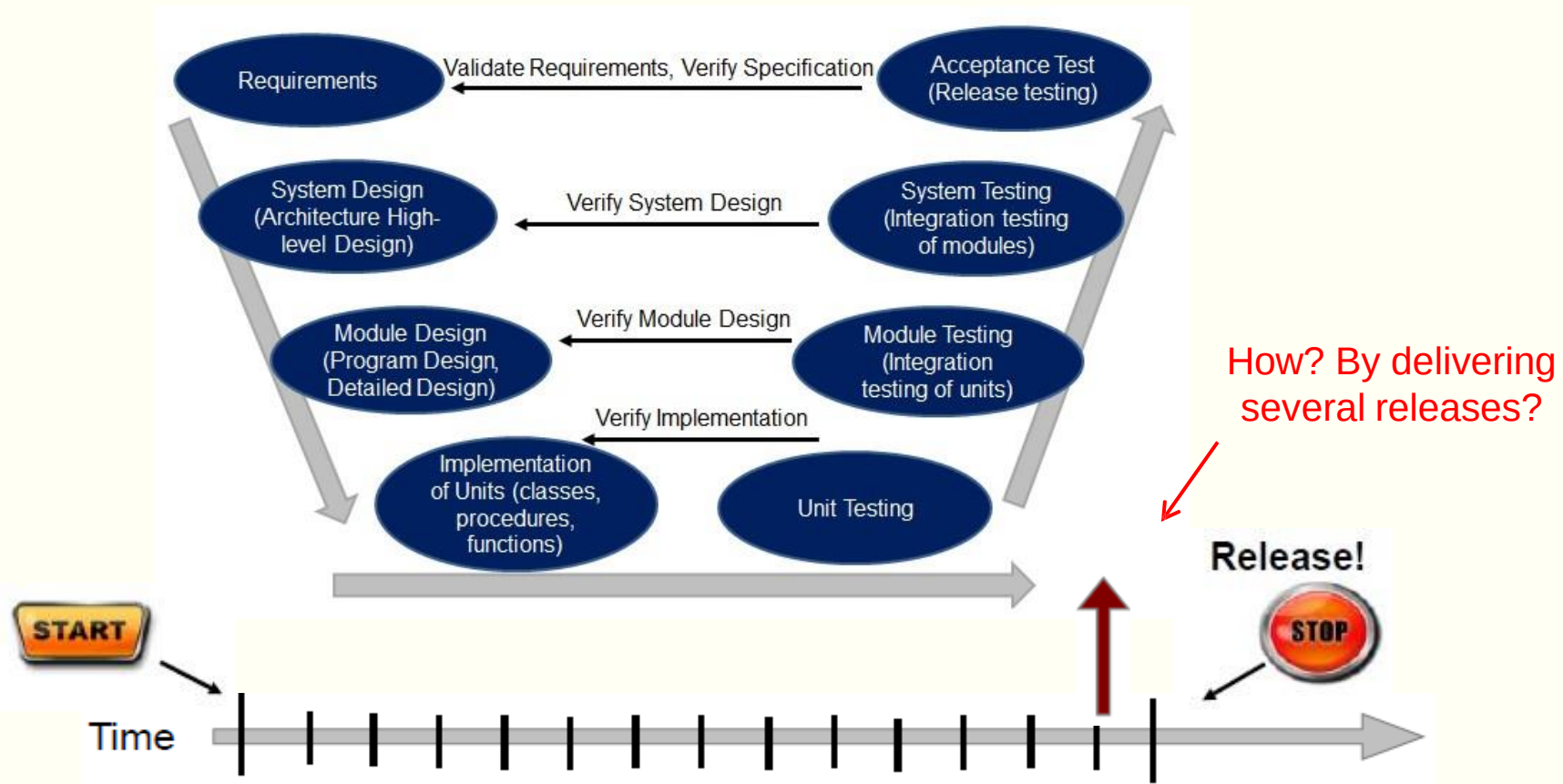


Incremental Development

- Incremental development **partitions a system by functionality**
- Early **release** starts with small, functional subsystem, later releases add functionality

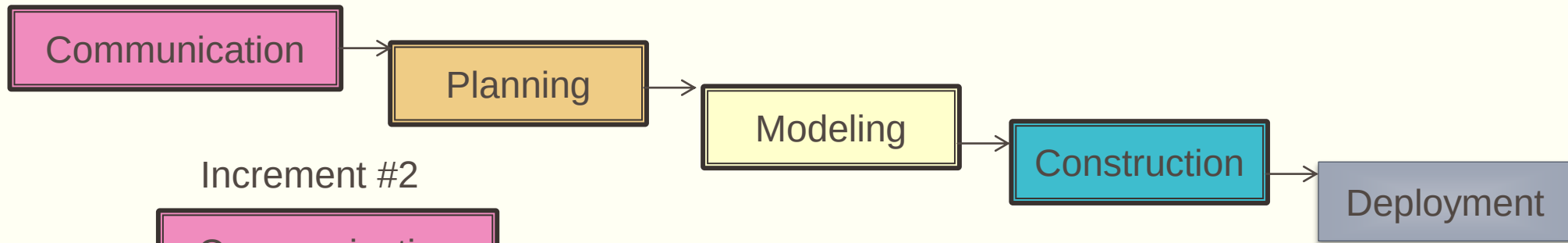


Incremental Development

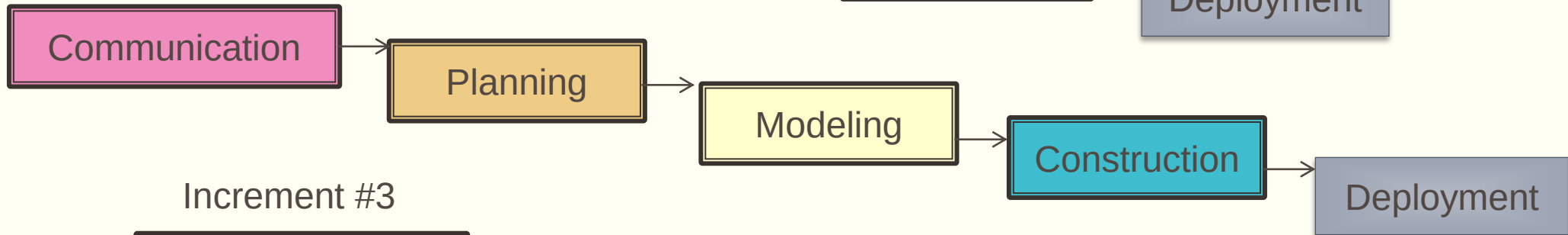


Incremental Model (Diagram)

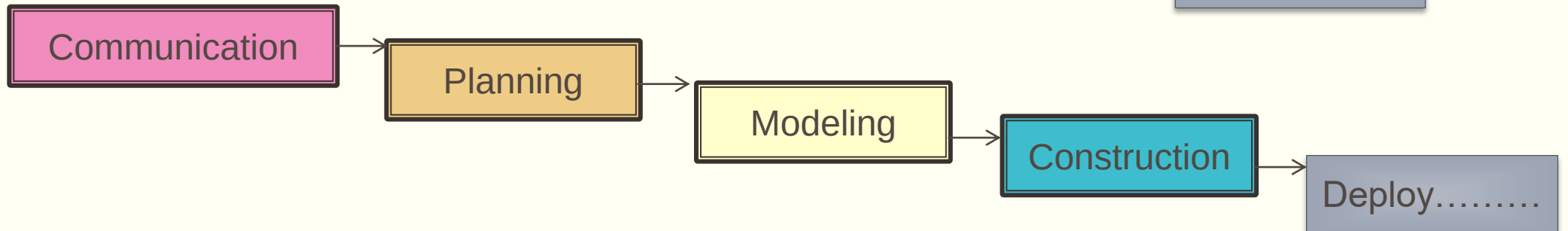
Increment #1



Increment #2



Increment #3

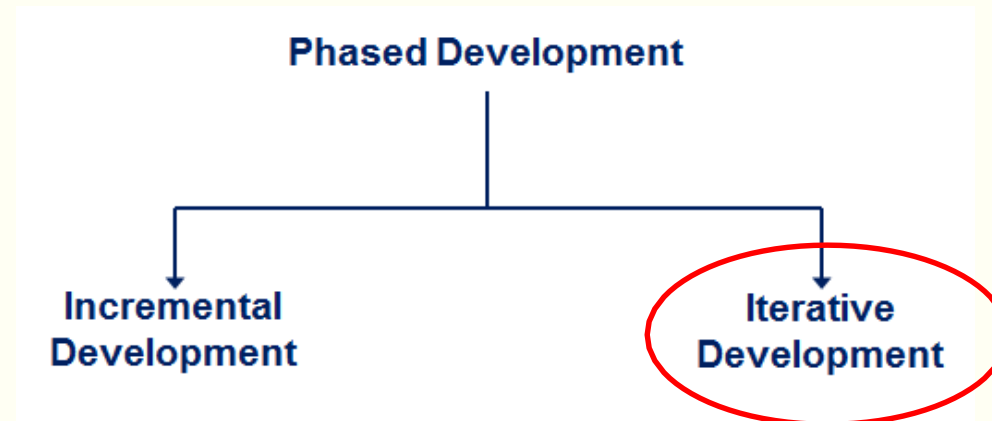


Incremental Model (Description)

- Used when **requirements** are **well understood**
- Multiple independent deliveries are identified
- Work flow is in a linear (i.e., sequential) fashion within an increment and is staggered between increments
- Iterative in nature; focuses on an operational product with each increment
- Provides a **needed set of functionality** sooner while delivering optional components later
- Useful also when staffing is too short for a full-scale development

Iterative Development

- Iterative development **improves** overall system in **each release**
- Delivers a **full system** in the **first release**, then **changes** the functionality of each subsystem with **each new release**



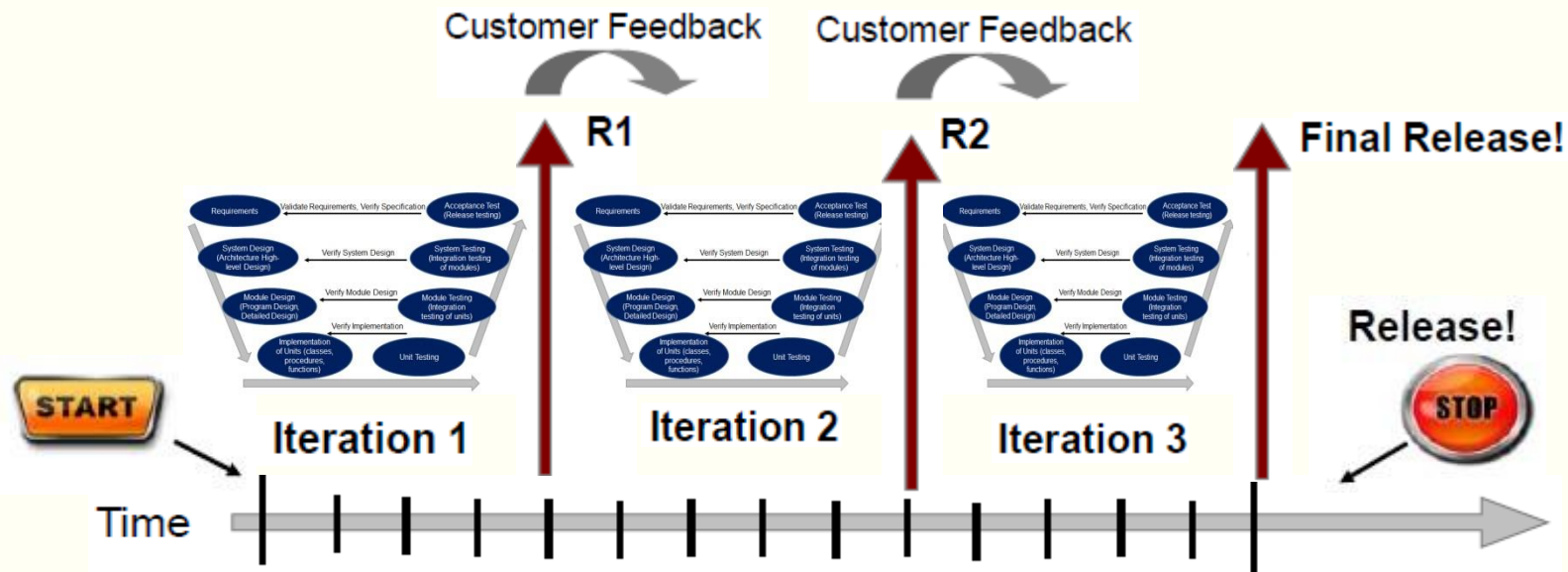
Iterative Development

When should the releases take place?

Time-boxing - The time period is fixed for each iteration.

When should the releases take place?

Prioritized functionality - Do the most important parts first.



Iterative vs. Incremental Development

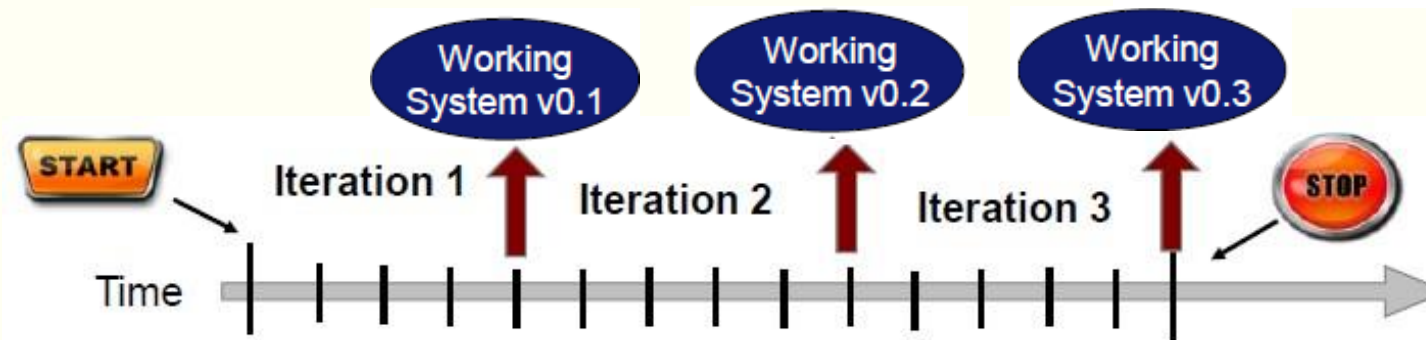
Incremental Development

Add a new **"part"** at each increment



Iterative Development

Improve a **"working system"** at each iteration



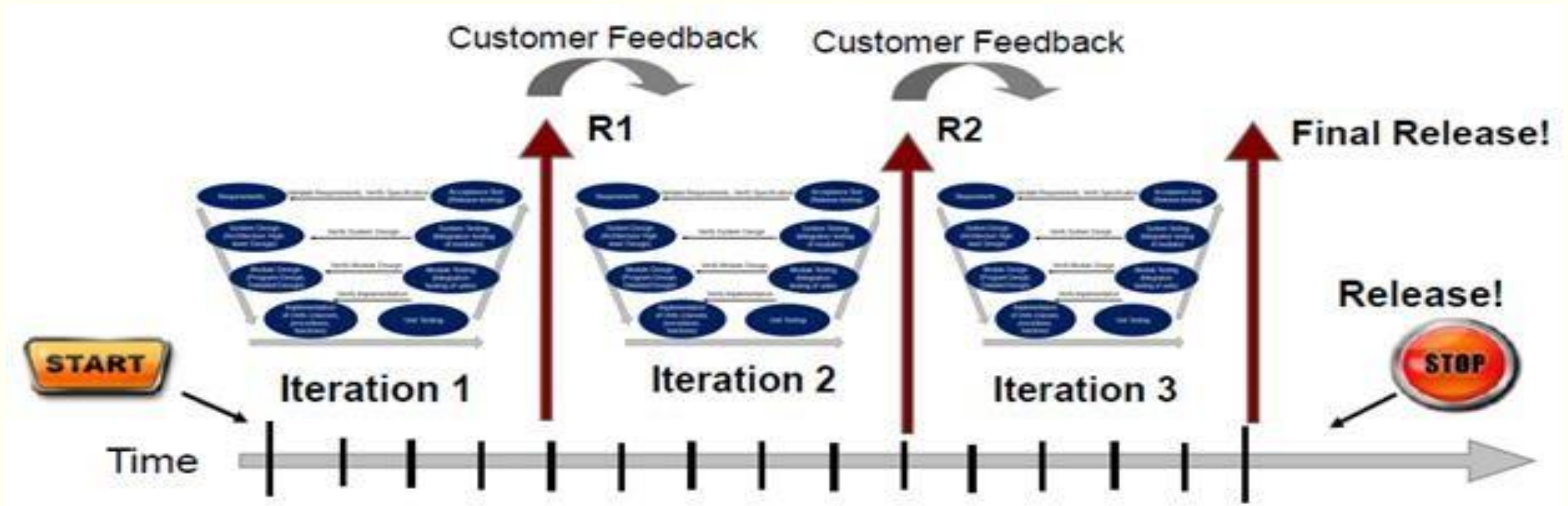
Note. Both concepts are often combined and sometimes misleading called just iterative development.

Iterative Development - Pros

- **Misunderstandings** and **inconsistency** are made **clear** early (e.g. between requirement, design, and implementation)
- Encourage to use feedback -> elicit the **real requirements**
- Forced to focus on the **most critical issues**
- Continuous testing offers **project assessment**
- **Workload** is spread out over time (especially test)
- The team can get "lesson learned" and continuously **improve the process**
- Stakeholders gets concrete **evidence of progress**

Iterative Development - Cons

- Problem with **current business contracts**, especially fixed-price contracts
- With **short iterations** it can be hard to **map** customer requirements to iterations



Incremental Development:



Iterative Development:

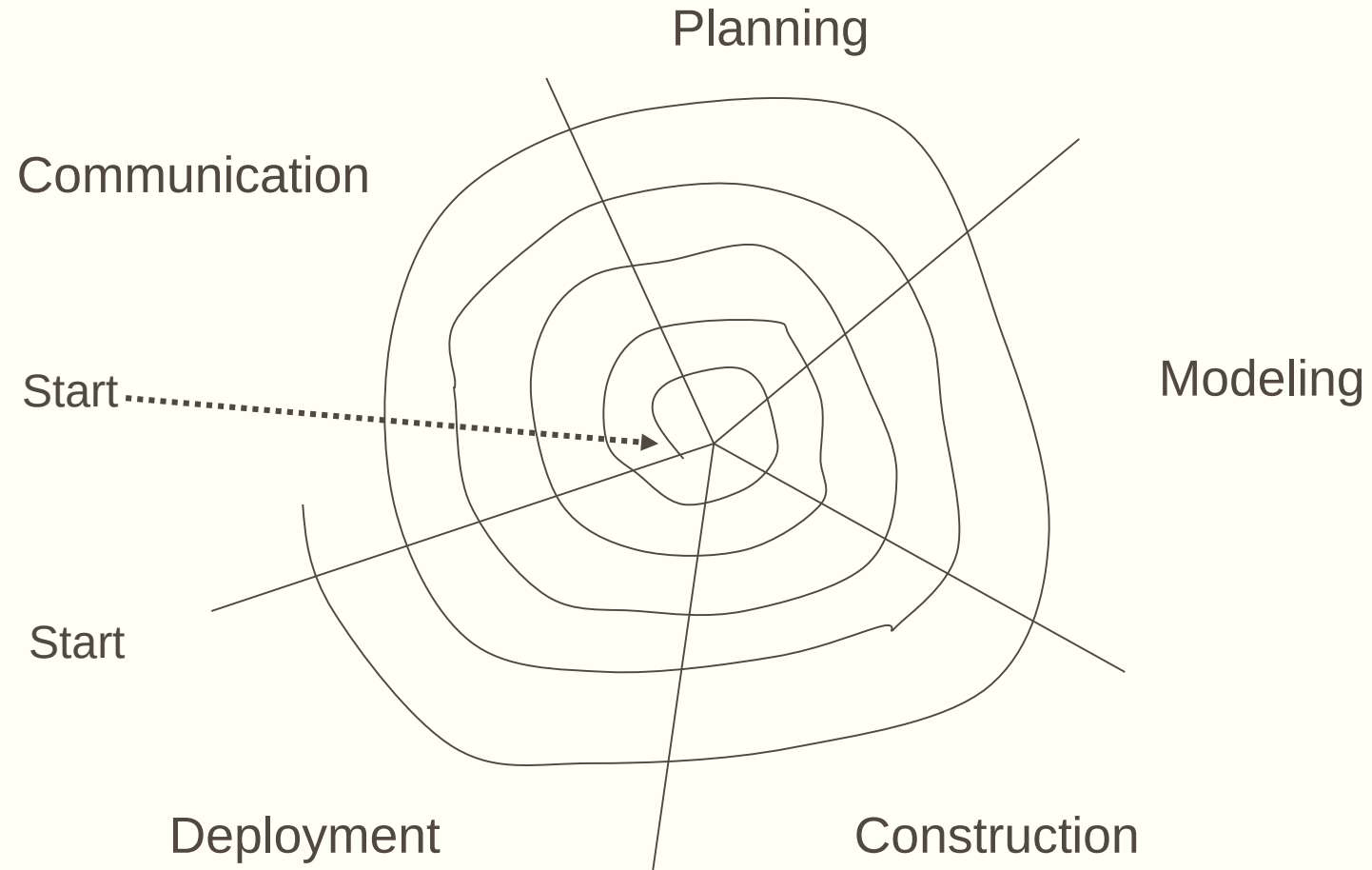


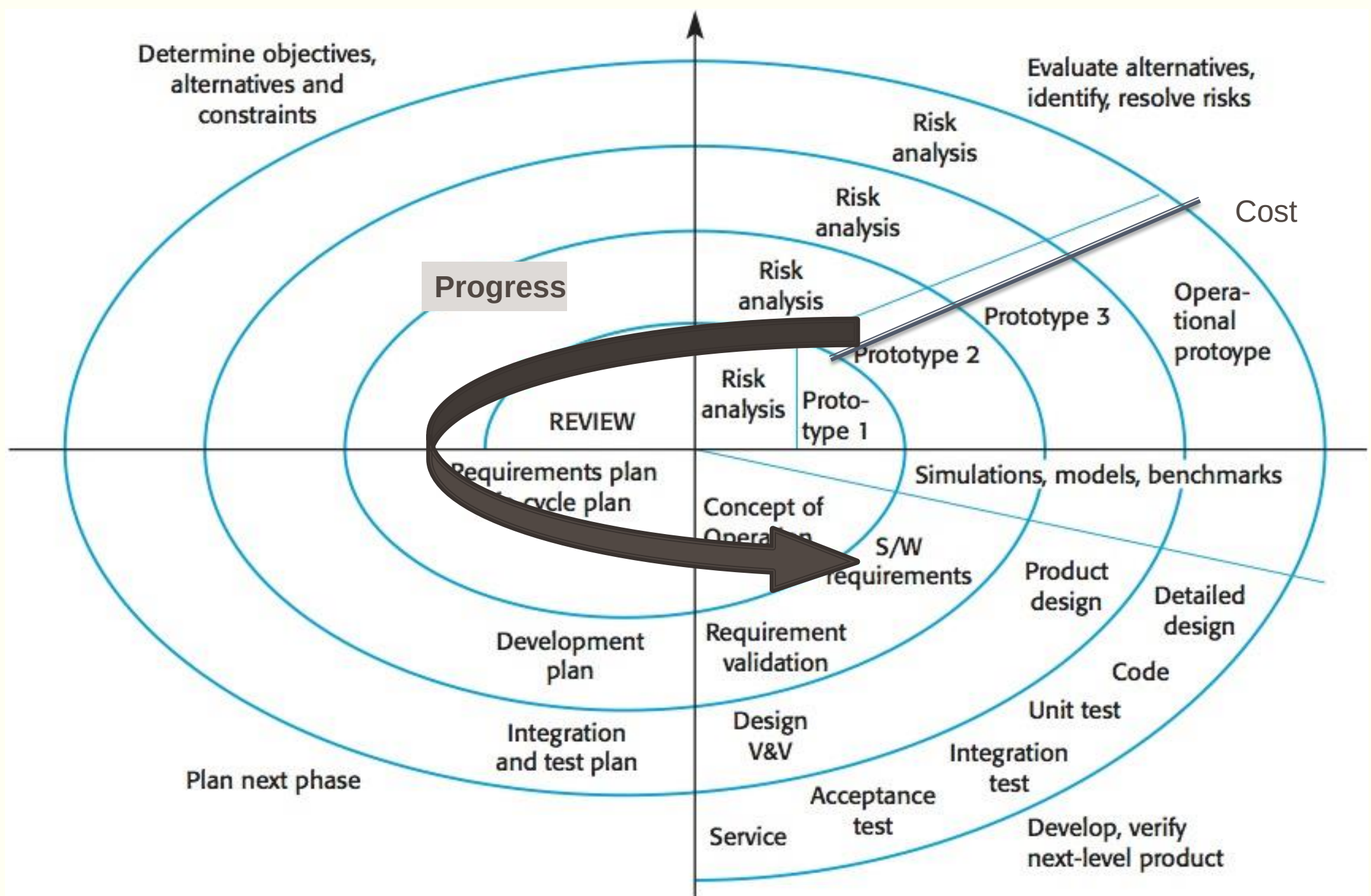
Activity

A customer wants to develop a word processing software package. It is estimated that the software package will be completed within one and half year. The customer wants that the software package should be develop before estimated time or he/she could be able to start working on the software package as soon as possible. What process model company should follow?



Spiral Model (Diagram)





Spiral Model (Description)

- Follows an evolutionary approach
- Used when requirements are not well understood and risks are high
- Inner spirals focus on identifying software requirements and project risks; may also incorporate prototyping
- Outer spirals take on a classical waterfall approach after requirements have been defined, but permit iterative growth of the software

-
-
- Operates as a risk-driven model...a go/no-go decision occurs after each complete spiral in order to react to risk determinations
 - Requires considerable expertise in risk assessment
 - Serves as a realistic model for large-scale software development

Prototyping Model



- A customer defines a set of general objectives for software but does not identify detailed **input**, **processing**, or **output** requirements



- The **developer** may be **unsure** of the **efficiency** of an algorithm, the **adaptability** of an operating system

Prototyping Model

- This model adds prototyping as **sub-process**
- A prototype is a **partially developed** product that enables customers and developers to examine some aspect of a proposed system and decide if it is suitable for a finished product

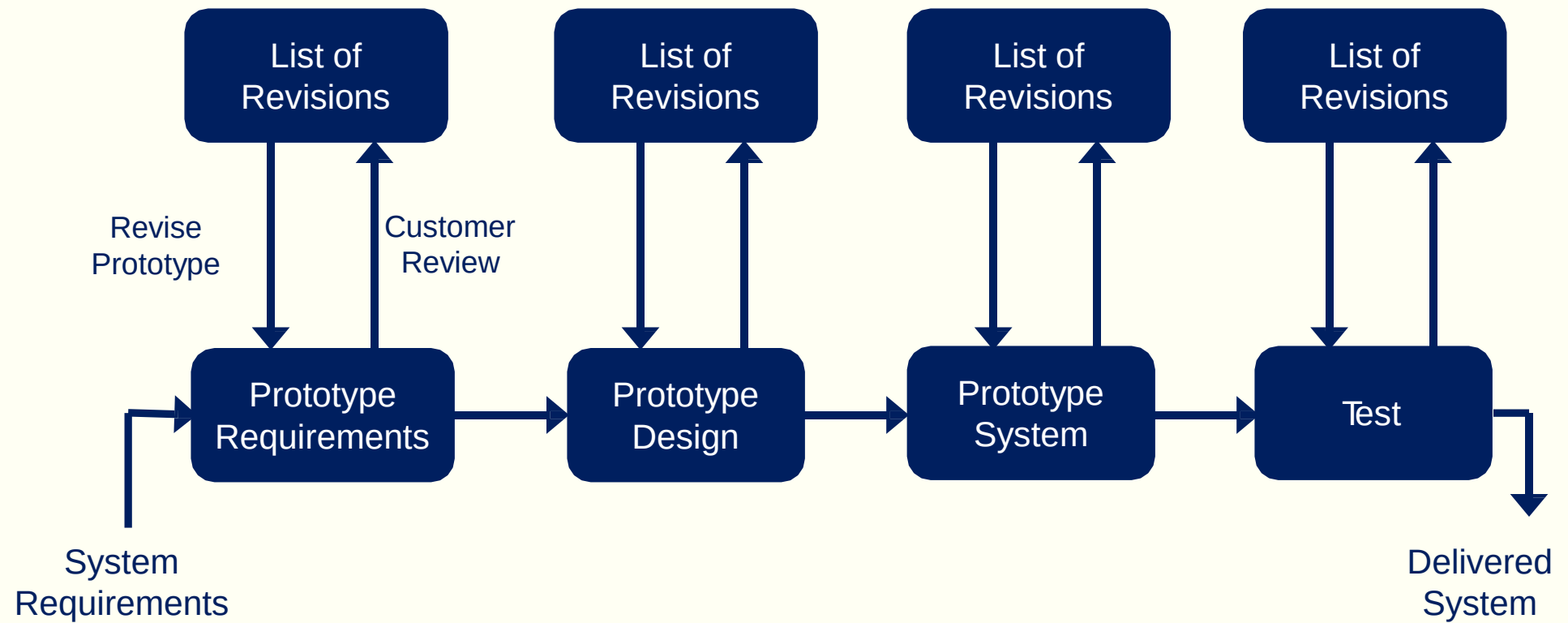


Prototyping Model

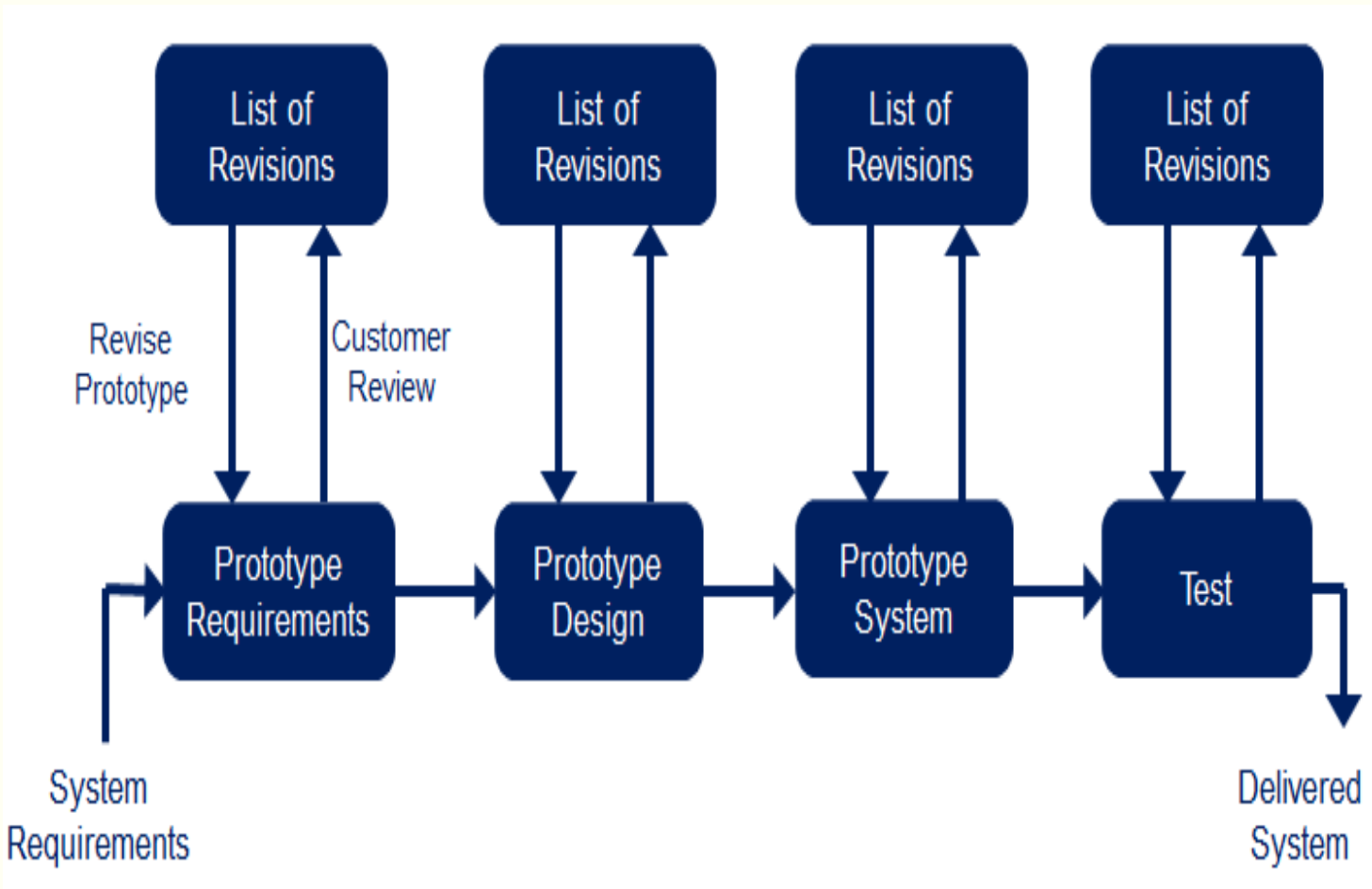
- Used to explore the risky aspects of the system
 - Risk of developing the “wrong” system
 - User interface without functionality
 - Technical risks
- Prototype may be **thrown away** or **evolve** into product



Prototyping Model

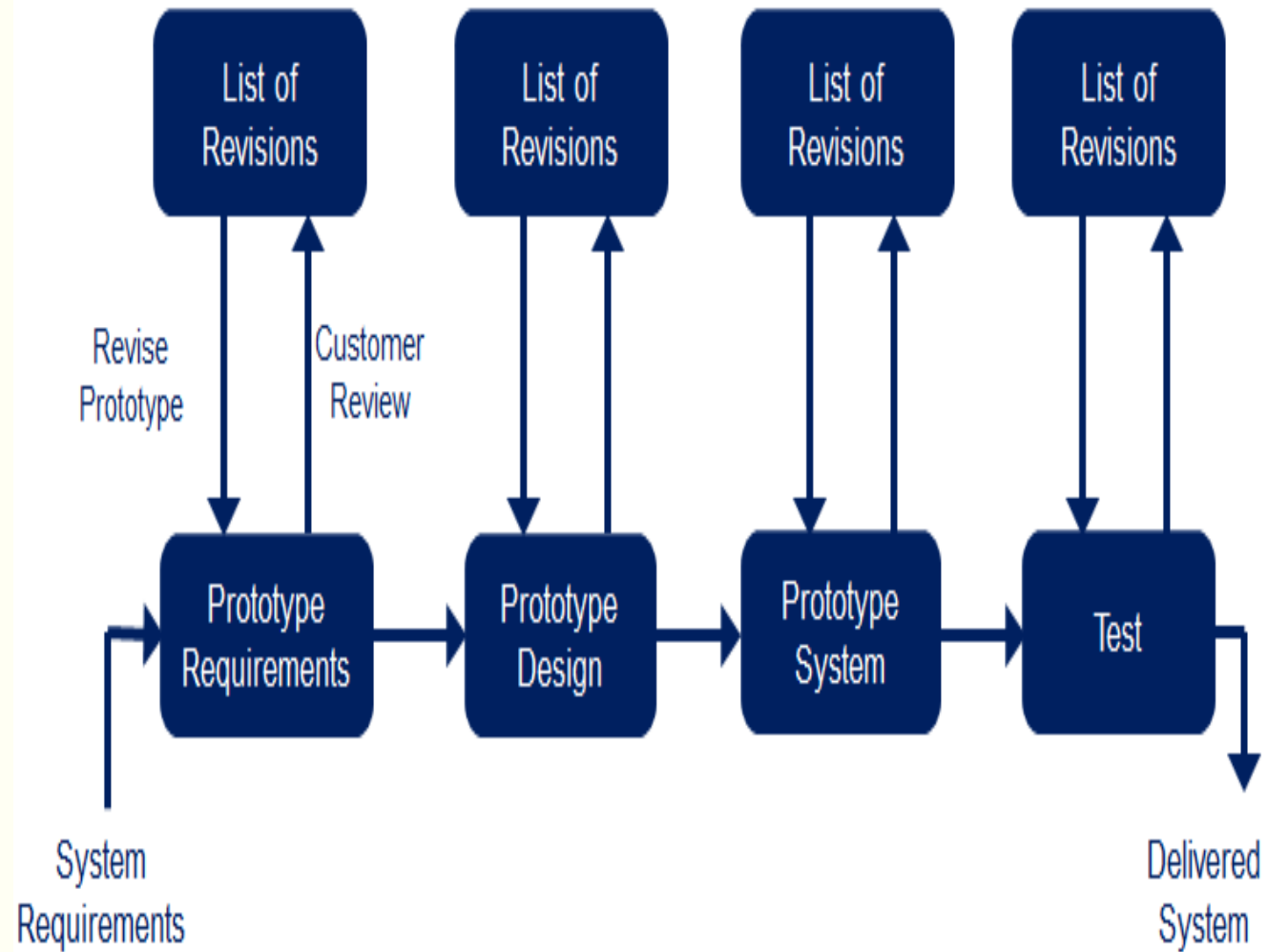


- able for **large systems** for
 ch there is no manual
 cess to define the
 uirements
- ch **feedback** from
 tomer
- ality** of software is good



Prototyping Model (**Cons**)

- It may be impossible to tune the prototype to meet **non-functional** requirements
- Lot of **time** at customer
- The **changes** made during prototype development will probably have **degraded** the system structure



Software Prototyping

- Software prototyping makes the **requirements** more **real** and **closes gaps** in understanding of the requirements
- Prototyping puts a **mock-up** of a new system
- **Early feedback** on prototypes helps the stakeholders arrive at a **shared understanding** of the system's requirements

Prototyping - Major Purposes

- **Clarify and complete the requirements:**

- Used as a requirements tool, the prototype is a preliminary **implementation** of a **part** of the **system** that's not well understood
- User evaluation of the prototype **points out problems** with the requirements, which you can correct at low cost before you construct the actual product

Prototyping - Major Purposes

- **Explore design alternatives:**

- A prototype lets stakeholders explore different **user interaction** techniques, optimize system **usability**, and evaluate potential technical approaches
- Prototypes can demonstrate requirements **feasibility** through working designs

Prototyping - Major Purposes

- **Grow into the ultimate product:**
 - A prototype is a **functional implementation** of an initial subset of the product, which can be elaborated into the **complete product** through a sequence of small-scale **development cycles**

Horizontal Prototypes

- Also called **behavioral prototype** or **mock-up**
- It doesn't dive into all the layers of an architecture or into system details but **depicts a portion of the user interface**
- This type of prototype lets you explore some **specific behaviors** of the intended system, with the goal of refining the requirements

Vertical Prototypes

- Also known as **Structural Prototype**
- It implements a **slice** of application functionality from the **user interface** through the technical **services layers**
- A vertical prototype works like the **real system** is supposed to work because it **touches** on **all levels** of the **system implementation**

Vertical Prototypes

- Develop a vertical prototype when you're **uncertain** whether a proposed **architectural** approach is **feasible** or not

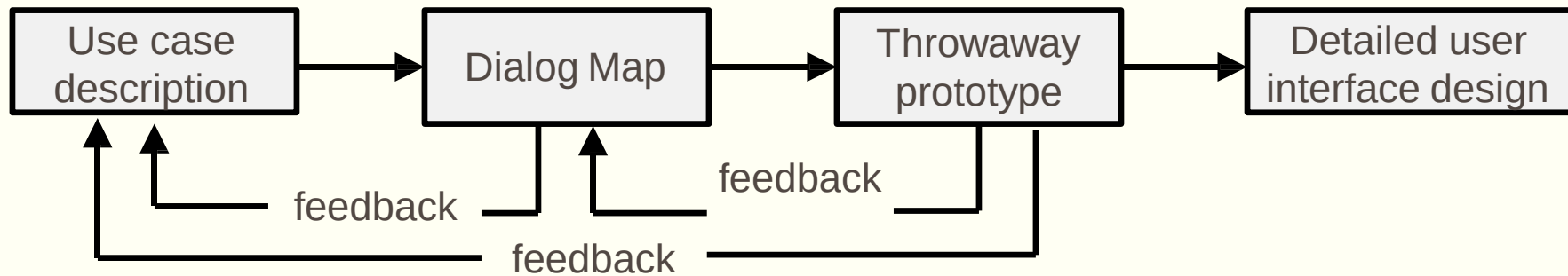
Throwaway Prototypes

- Build a **throwaway prototype** (or **exploratory prototype**) to **answer questions**, **resolve uncertainties**, and **improve requirements quality**
- Throwaway prototype emphasizes **quick implementation** and **modification** over **robustness**, **reliability**, **performance**, and **long-term maintainability**

Throwaway Prototypes

- The throwaway prototype is most **appropriate** when the team faces uncertainty, ambiguity, incompleteness, or vagueness in the requirements
- A prototype that helps users and developers visualize how the requirements might be implemented can reveal gaps in the requirements

Throwaway Prototypes



Activity sequence from use cases to user interface design using a throwaway prototype

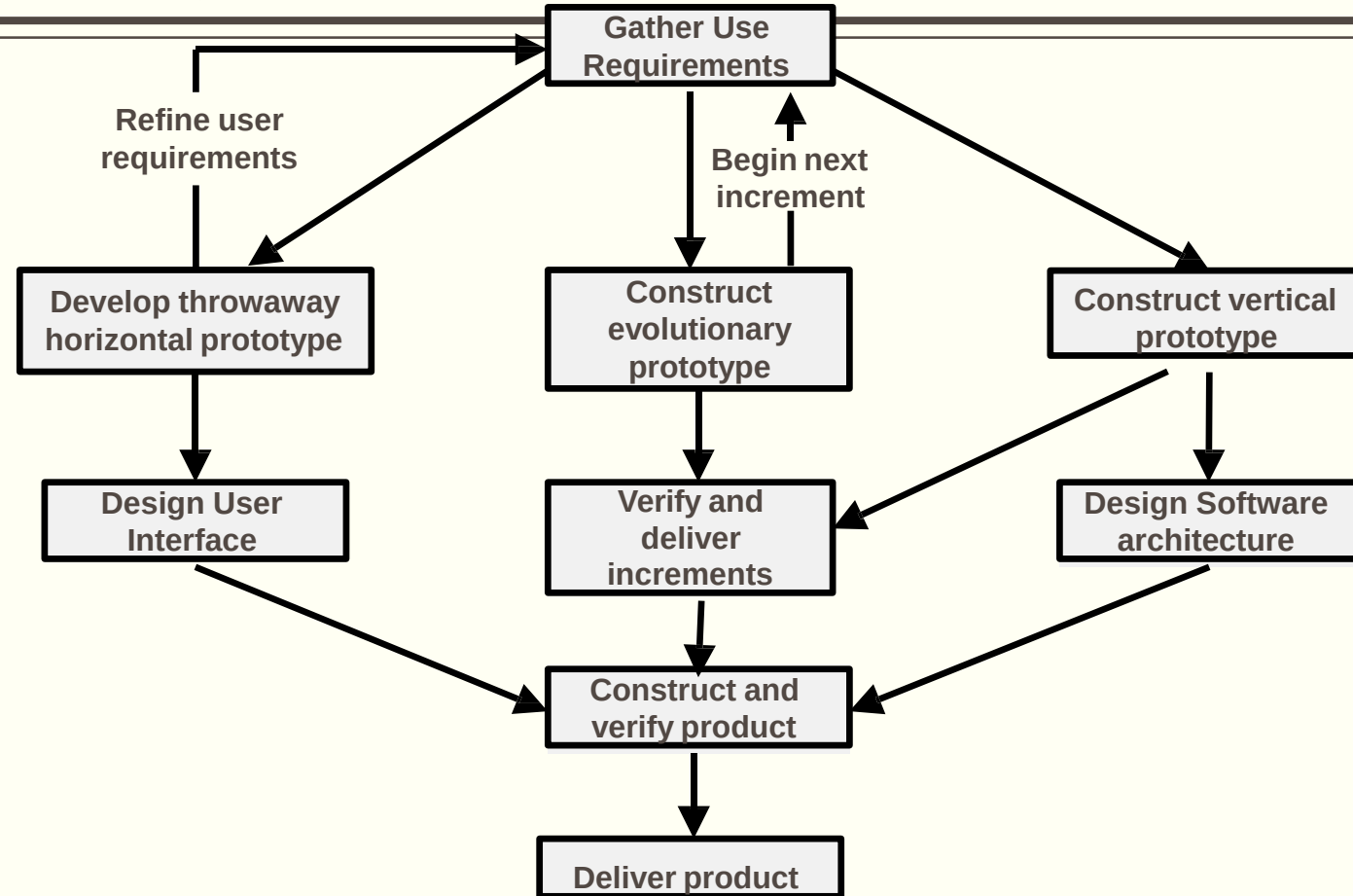
Evolutionary Prototypes

- An evolutionary prototype provides a solid architectural foundation for building the product incrementally as the requirements become clear over time
- Evolutionary prototyping is a component of the spiral software development life cycle model and of some object-oriented software development processes

Evolutionary Prototypes

- In contrast to the quick-and-dirty nature of throwaway prototyping, an evolutionary prototype must be built with robust, production-quality code from the outset
- An evolutionary prototype must be designed for **easy growth** and **frequent enhancement**, so developers must emphasize software architecture and solid design principles

Gather Use	
1	



Several possible ways to incorporate prototyping into the software development process